

Relazione Progetto: File System Statistics (FSStat)

Programmazione Concorrente e Distribuita

Alessandro Martini, Alessandro Torelli

Anno Accademico 2025/2026

Sommario

Il presente documento descrive la progettazione e l'implementazione della libreria **FSStatLib**, sviluppata in Java 25. L'obiettivo del progetto è analizzare un albero di directory del file system, calcolando il numero totale di file e la loro distribuzione in fasce di dimensione. L'elaborazione è stata parallelizzata utilizzando tre metodi e tecnologie differenti: i Virtual Threads, programmazione asincrona basata su event loop, e la programmazione reattiva con RX.

Indice

1	Introduzione e Requisiti	2
2	Analisi del Problema e Scelte Concorrenti	2
2.1	Approccio con i Virtual Threads	2
2.2	Approccio Asincrono basato su Event Loop	3
2.3	Approccio Reattivo con RX	3
3	Architettura e Design	4
3.1	Design dell'implementazione Virtual Threads	4
3.2	Design dell'implementazione Event Loop	4
3.3	Design dell'implementazione RX	4
4	Risultati e Osservazioni	4

1 Introduzione e Requisiti

Il progetto prevede la creazione di una libreria asincrona in grado di esplorare il file system a partire da una directory radice e fornire un report statistico (numero totale di file e raggruppamento in bande in base alla dimensione).

Oltre al requisito base, è stato implementato il requisito opzionale che prevede la possibilità di fermare la computazione in qualsiasi momento e di ricevere aggiornamenti in tempo reale sulle statistiche.

2 Analisi del Problema e Scelte Concorrenti

L'attraversamento ricorsivo di un file system e l'interrogazione delle proprietà dei file sono tipiche operazioni legate all'I/O. In questi scenari di I/O-bound, l'uso intensivo dei classici thread di piattaforma presenta gravi limiti di scalabilità. Per esplorare soluzioni diverse a questo problema di concorrenza, il progetto è stato declinato in tre paradigmi distinti.

2.1 Approccio con i Virtual Threads

I Virtual Threads, introdotti stabilmente in Java 21 (JEP 444), sono thread *user-mode* gestiti dalla JVM anziché dal sistema operativo. Rispetto ai *platform thread*, il loro overhead di creazione e il consumo di memoria sono trascurabili, il che li rende adatti a un modello *Thread-per-task*: è possibile istanziarne decine di migliaia senza incorrere in `OutOfMemoryError` né in degrado delle prestazioni.

Questa caratteristica si adatta in modo naturale all'attraversamento ricorsivo di un albero di directory: la struttura del file system è intrinsecamente ramificata e imprevedibile, e ogni nodo (directory) richiede operazioni di I/O bloccanti a latenza variabile (apertura dello stream, lettura degli attributi dei file con `Files.size()`). Assegnare un Virtual Thread a ciascun nodo permette di sfruttare appieno la concorrenza senza dover dimensionare a priori un pool.

Struttura dell'implementazione. Il punto di ingresso pubblico è il metodo `getFSReport()`, che restituisce una `CompletableFuture<FSReport>` per offrire un'interfaccia non bloccante al chiamante (CLI o GUI). Internamente, la computazione reale viene avviata tramite `CompletableFuture.supplyAsync()` che esegue il corpo su un Virtual Thread. All'interno di questo contesto viene creato un `ExecutorService` dedicato tramite `Executors.newVirtualThreadPerTaskExecutor()`, racchiuso in un blocco `try-with-resources`:

```
1 try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
2     Path rootPath = Paths.get(directoryPath);
3     return processDirectory(rootPath, maxFS, nb, executor).get();
4 }
```

La chiusura automatica dell'executor al termine del blocco `try` ha una semantica cruciale: `ExecutorService.close()` invoca `shutdown()` seguito da `awaitTermination()`, garantendo che nessun task figlio venga abbandonato prima che il risultato finale sia disponibile. Questo meccanismo funge da *barriera di sincronizzazione* implicita su tutto l'albero dei task.

Ricorsione concorrente e Map-Reduce. Il metodo `processDirectory()` incarna il pattern *divide et impera*: per ogni nodo dell'albero viene sottomesso all'executor un nuovo task (e quindi un nuovo Virtual Thread). All'interno del task, si itera sul contenuto della directory tramite `Files.newDirectoryStream()`:

- Per ogni **file regolare**: si invoca `Files.size()` (operazione I/O bloccante) e si aggiorna un oggetto `FSReport locale` al Virtual Thread corrente.

- Per ogni **sottodirectory**: si sottomette ricorsivamente un nuovo task all'executor, raccogliendo il `Future<FSReport>` restituito in una lista di *subTask*.

Terminata la scansione della directory corrente, il Virtual Thread chiama `task.get()` su ciascun `subTask` per attenderne il completamento e fondere i risultati parziali nel report locale tramite il metodo `FSReport.merge()`:

```
1 for (Future<FSReport> task : subTasks) {
2     localReport.merge(task.get());
3 }
```

Questa fase di riduzione segue un pattern *Map-Reduce*: la fase di *map* distribuisce il lavoro di scansione su Virtual Thread indipendenti, mentre la fase di *reduce* aggrega i report parziali bottom-up lungo l'albero, propagando i risultati verso il nodo radice.

Assenza di contesa e gestione dell'I/O bloccante. Una proprietà rilevante di questo design è la completa assenza di stato condiviso mutabile tra Virtual Thread concorrenti: ogni task opera esclusivamente sul proprio `FSReport` locale. La fusione dei risultati (`merge`) avviene sempre in modo sequenziale nel Virtual Thread padre, dopo che i figli sono terminati. Ne consegue che non è necessario alcun meccanismo di sincronizzazione esplicito (né `synchronized` né `ReentrantLock`) per proteggere il report.

Quando un Virtual Thread si blocca su un'operazione di I/O — ad esempio `Files.size()` o `task.get()` in attesa di un risultato non ancora disponibile — la JVM effettua automaticamente l'*unmount* del Virtual Thread dal *carrier thread* (il platform thread sottostante), che viene immediatamente liberato per eseguire un altro Virtual Thread pronto. Questo meccanismo permette al ridotto pool di carrier thread (tipicamente uguale al numero di core fisici) di servire migliaia di Virtual Thread concorrenti, eliminando il costo del context switch a livello di sistema operativo.

2.2 Approccio Asincrono basato su Event Loop

Il secondo paradigma adottato si basa sull'Event Loop (tipico di ecosistemi come Node.js o framework Java come Vert.x). In questo modello, un numero limitato di thread (spesso coincidente con il numero di core della CPU) esegue un ciclo continuo di dispacciamento degli eventi. Per non bloccare mai il thread dell'Event Loop, l'esplorazione del file system viene effettuata tramite API non bloccanti (NIO.2) e l'esecuzione prosegue tramite *callback* o concatenazione di `CompletableFuture`. Il carico di lavoro viene quindi frammentato in piccoli task asincroni che si risvegliano solo quando il dato (es. la dimensione del file) è pronto dal disco, garantendo alta concorrenza senza il sovraccarico del context switch tra migliaia di thread fisici.

2.3 Approccio Reattivo con RX

La terza tecnologia utilizza la Programmazione Reattiva (es. RxJava), un paradigma orientato ai flussi di dati (Data Streams) e alla propagazione dei cambiamenti. L'attraversamento del file system è stato modellato come uno stream asincrono di eventi (file trovati o directory aperte) emessi da un *Publisher / Observable*. L'approccio reattivo ha permesso di operare in modo puramente dichiarativo: attraverso operatori funzionali (`map`, `flatMap`, `filter`, `scan`) gli eventi vengono trasformati e aggregati al volo per costruire il report. Questo approccio è intrinsecamente non bloccante e facilita enormemente la gestione degli aggiornamenti in tempo reale per la GUI.

3 Architettura e Design

Il progetto è suddiviso seguendo il principio di Separazione delle Responsabilità (Separation of Concerns). Oltre al livello dell'interfaccia (CLI e GUI), la logica di dominio (il package `lib`) è stata strutturata per accomodare le tre diverse implementazioni tramite un'interfaccia comune, permettendo al chiamante di scegliere il motore di concorrenza preferito.

3.1 Design dell'implementazione Virtual Threads

L'architettura con Virtual Threads mantiene uno stile di programmazione imperativo e sequenziale, molto intuitivo da scrivere e da leggere. L'ingresso avviene sottomettendo i task a un `Executors.newVirtualThreadPerTaskExecutor()` [cite: 1]. L'aggregazione dei risultati sfrutta le chiamate bloccanti naturali (`Future.get()`), che risultano sicure e altamente scalabili proprio grazie alla meccanica di *unmounting* [cite: 1]. L'interruzione dinamica (stop) viene propagata controllando periodicamente un flag `volatile` condiviso.

3.2 Design dell'implementazione Event Loop

L'architettura in questo caso sfrutta pesantemente le chiusure (closures) e il concatenamento asincrono. L'esplorazione utilizza metodi ricorsivi che restituiscono `CompletableFuture`, elaborati su un pool ristretto (`ForkJoinPool.commonPool` o un executor custom). A differenza dei Virtual Threads, lo stato condiviso per il report può essere gestito spesso senza lock pesanti, incapsulando gli aggiornamenti direttamente nei thread del loop o riducendo i risultati alla fine della catena (*Map-Reduce* puramente funzionale). Il meccanismo di stop si basa sull'interruzione della propagazione delle callback.

3.3 Design dell'implementazione RX

Il design reattivo stravolge l'architettura passando dalla "richiesta di dati" (pull) alla "reazione agli eventi" (push). La libreria espone un `Flowable<FSReport>` o `Observable<FSReport>`.

- **Aggiornamenti Dinamici:** La GUI si limita a effettuare una `subscribe()` sullo stream, ricevendo un nuovo snapshot del report ogniqualvolta la pipeline reattiva emette un aggiornamento.
- **Cancellazione:** Per fermare il task è sufficiente invocare il metodo `dispose()` (o disiscriversi dallo stream); il framework RX propaga istantaneamente e in modo automatico il segnale di interruzione alla sorgente (l'esploratore delle directory), chiudendo pulitamente le risorse aperte.

4 Risultati e Osservazioni

Dai test effettuati localmente, l'overhead della creazione di migliaia di Virtual Threads è risultato impercettibile, permettendo all'applicazione di consumare una frazione della RAM che sarebbe stata necessaria usando un pool cached di platform threads [Virtual Threads.pdf]. Il sistema è risultato stabile, senza mai incorrere in `OutOfMemoryError`, confermando le tesi sulla scalabilità di questo nuovo paradigma su workload I/O-bound.